Deep Dive: Quantizing LLMs

Companion videos: https://youtu.be/kw7S-3s50uk + https://youtu.be/fXBBwCIA0Ds

Julien Simon

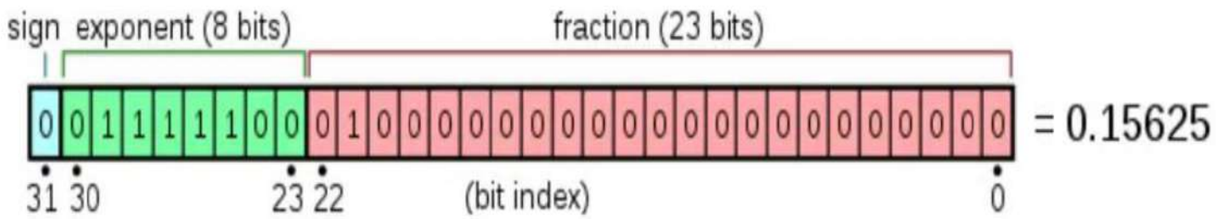
https://www.linkedin.com/in/juliensimon

https://www.youtube.com/juliensimonfr

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

What is quantization?

- Model weights are learned during training, and stored as numerical values
- Common data types: FP32, FP16, BF16

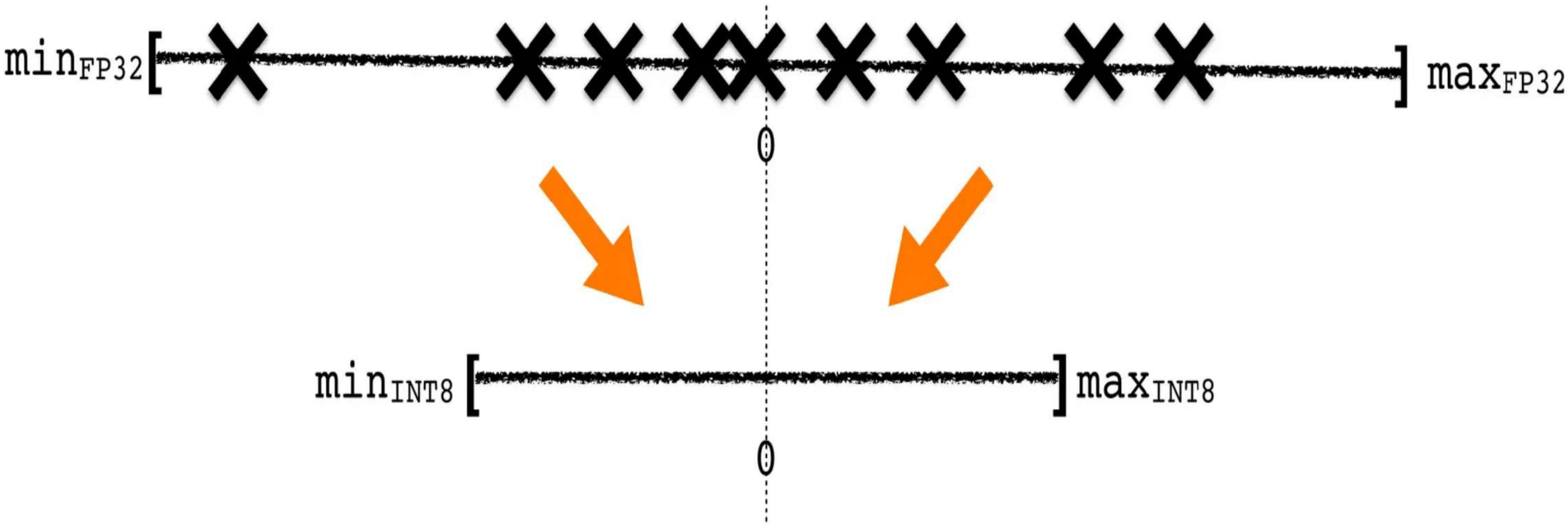


- The larger the data type, the finer the granularity: models are more accurate
- This comes at the cost of higher memory and compute requirements: models are larger, inference is slower
- The purpose of quantization is to rescale weights and/or activations to shorter data types, to reduce memory and compute requirements while minimizing loss of accuracy
- Common lower-precision data types: INT8, INT4

How can we best map a high-precision data format to a lower-precision format?

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Rescaling weights and activations



Note: only one dimension is shown,
but we need to do the same on all tensor dimensions!

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Mapping function

The mapping function maps the high-precision numerical space to a lower-precision space

This usually takes place per layer, but finer-grained quantization is also possible (e.g. per row or per column)

$$Q(r) = round\left(\frac{r}{S} + Z\right)$$

S,Z: quantization parameters

$$S = \frac{\beta - \alpha}{\beta_q - \alpha_q}$$

Ratio of the $[\alpha,\beta]$ input range to the $[\alpha_q,\beta_q]$ output range

For 8-bit quantization: $\beta_q - \alpha_q = 2^8 - 1$

$$Z = \alpha_q - \frac{\alpha}{S}$$

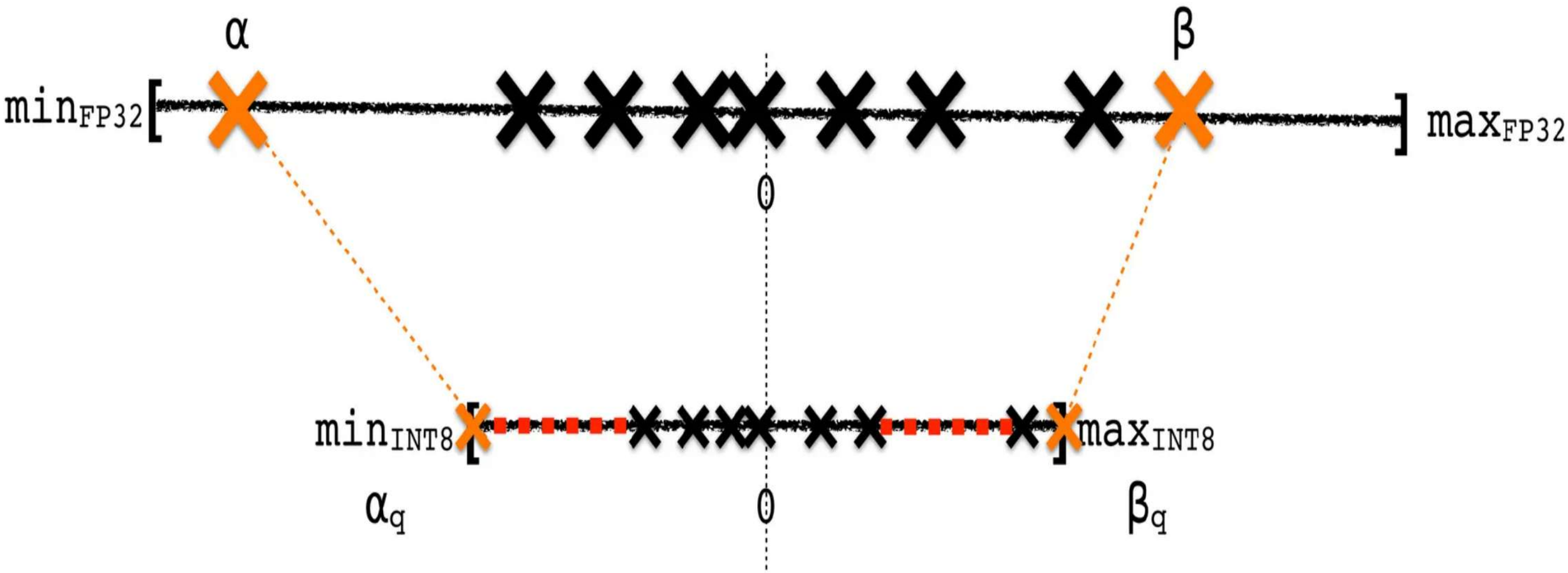
Bias value to map a zero input to a zero output

How can we best pick the $[\alpha,\beta]$ range to minimize quantization error?

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Picking the [α,β] range

	Range	Min. positive value
FP32 / BF16	[±1.18e-38, ±3.4e38]	1.4e-45
INT8	[-128, +127]	1



A simple technique is to use the minimum and maximum value of the input range

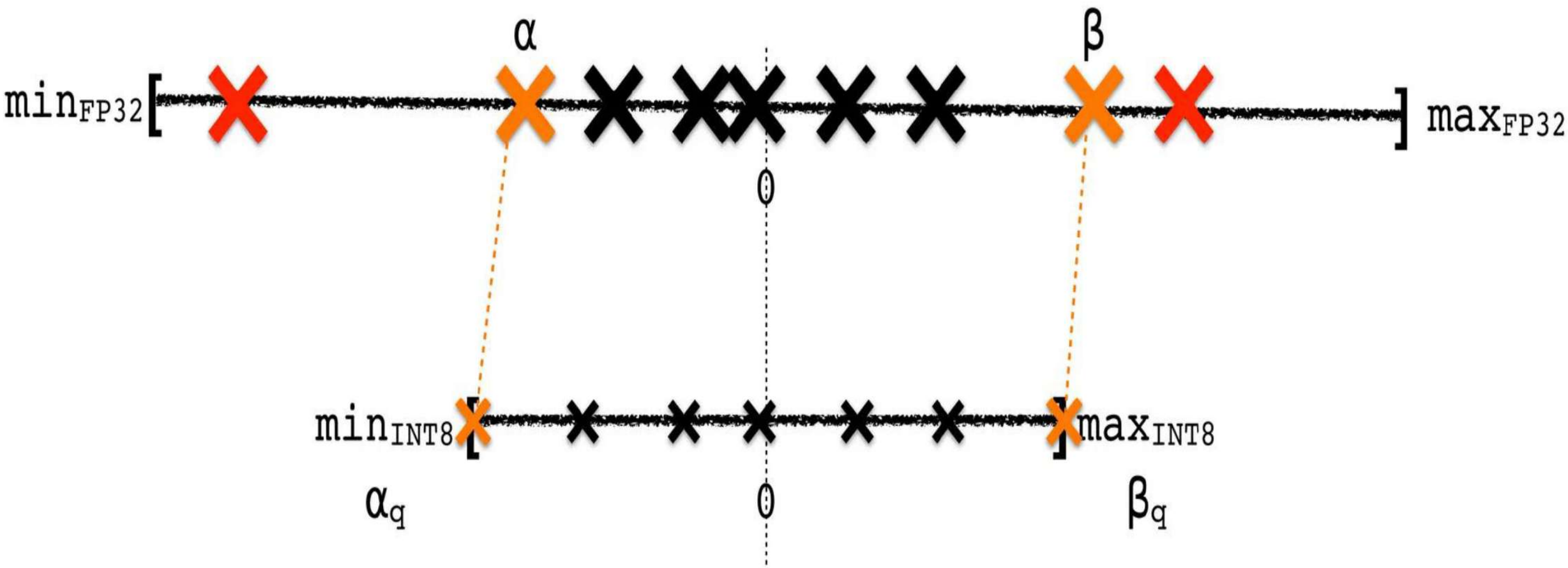
This is sensitive to outliers: part of $[\alpha_q,\beta_q]$ could be unused, and "squeeze" other values

Alternative: use percentiles or histogram bins

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Picking the [α,β] range

	Range	Min. positive value
FP32 / BF16	[±1.18e-38, ±3.4e38]	1.4e-45
INT8	[-128, +127]	1



To make the best use of $[\alpha_q,\beta_q]$, we need to get rid of outliers

How many can we eliminate without hurting accuracy too much?

We need to minimize information loss between the two distributions

➡️ Use a calibration dataset and minimize the Kullback-Leibler (KL) divergence 😱

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

When can we apply quantization?

- Post-training dynamic quantization (dynamic PTQ)
  - Load the trained model for inference and convert model weights ahead of time
  - Rescale activations on the fly, just before running the computation
  - The scale factor is computed dynamically 👍 simple, more flexibility 👎 some overhead
- Post-training static quantization (static PTQ)
  - Load the trained model for inference and convert model weights ahead of time
  - Predict a calibration dataset, and use the observed distribution of activations to set the scale factor
  - 👍 no overhead, lower latency 👎 calibration may not work well for all datasets, need different models
- Quantization-aware training (QAT)
  - Training still takes place with full precision
  - Weights and activations are "fake quantized" during training
  - 👍 higher quality models 👎 need to retrain, expensive for LLMs

- At inference time, a dequantization process returns results in the original data type

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Dynamic PTQ with PyTorch

https://pytorch.org/tutorials/intermediate/dynamic_quantization_bert_tutorial.html

```
import torch
from transformers import BertTokenizer, BertModel

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
orig_model = BertModel.from_pretrained("bert-base-uncased").to(device=device)

quantized_model = torch.quantization.quantize_dynamic(
    orig_model, {torch.nn.Linear}, dtype=torch.qint8
)
```

Quantization of the Linear layers to INT8

Model size: 437MB ➡️ 181MB (-58%)

Inference latency (1 thread): 34.7 ms ➡️ 24 ms (-31%)

<1% F1 degradation on MRPC task

```
(output): BertOutput(
          (dense): DynamicQuantizedLinear(in_features=3072,
out_features=768, dtype=torch.qint8, qscheme=torch.per_tensor_affine)
          (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
```

Amazon EC2 c6i.4xlarge, AWS Deep Learning AMI, PyTorch 2.2.0 + IPEX 2.2.0

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

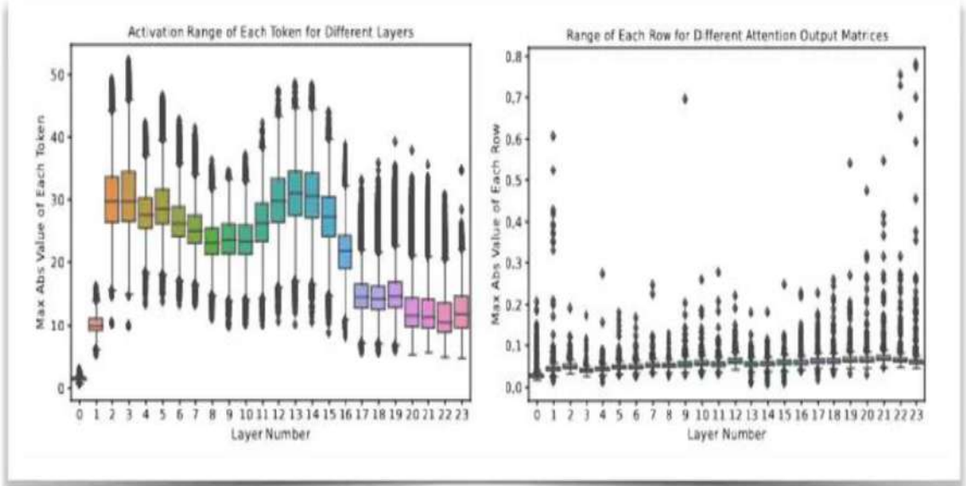
ZeroQuant

https://arxiv.org/abs/2206.01861 (06/2022) + https://github.com/mit-han-lab/smoothquant

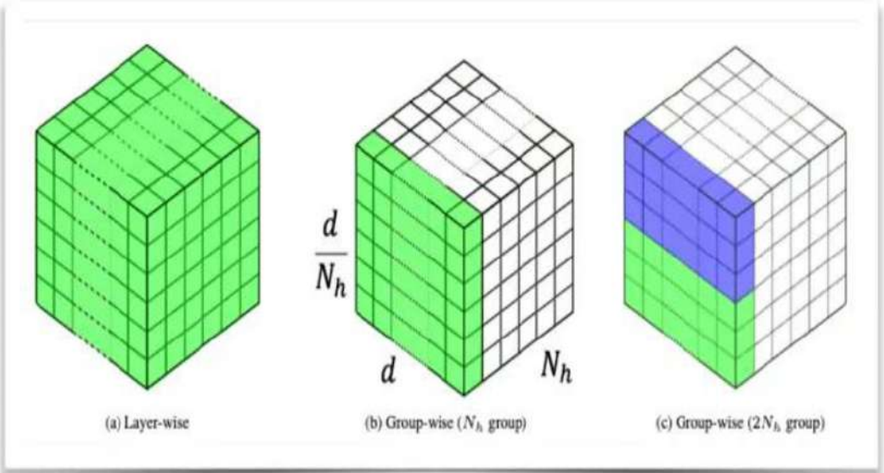
- First (?) dynamic post-training quantization optimized for LLMs
- INT8 weights and activations, or INT4 weights and INT8 activations
- Weight ranges are very different across layers, activation ranges even more so
- Group-wise quantization for weights https://arxiv.org/abs/1909.05840 (09/2019) partition tensors into weight groups, and quantize them separately
- Token-wise quantization for activations: dynamically compute the min/max range for each token
- Layer-by-layer knowledge distillation (LKD) for increased accuracy (optional)
- Optimized implementations for GPU hardware
- 2x-5x speedup (BERT), negligible accuracy drop for W8A8
- Stronger quantization has a larger impact, particularly on generative tasks

Precision (Method)	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)	Time Cost
W16A16	61.3	71.4	33.6	53.1	82.6	42.4	15.3	N/A
W8A8 (PTQ)	54.8	67.7	16.6	54.5	75.7	40.5	18.9	13 mins
W8A8 (ZeroQuant)	62.6	70.7	33.4	52.7	80.9	42.3	15.7	0
W4/8A16 (PTQ)	0.00	50.4	27.0	50.9	15.8	29.0	1.35e5	13 mins
W4/8A16 (ZeroQuant)	43.9	66.5	30.0	52.7	77.3	39.38	21.9	0
W4/8A16 (ZeroQuant-LKD)	59.4	69.5	31.6	52.7	79.7	41.5	17.6	3 hours
W4/8A8 (ZeroQuant)	46.8	66.4	28.8	52.7	76.2	39.24	24.1	0
W4/8A8 (ZeroQuant-LKD)	48.7	68.1	29.0	52.0	77.4	39.90	18.2	3 hours

GPT-3 1.3B



The activation range (left) and row-wise weight range of the attention output matrix (right) of different layers of GPT-3 350M



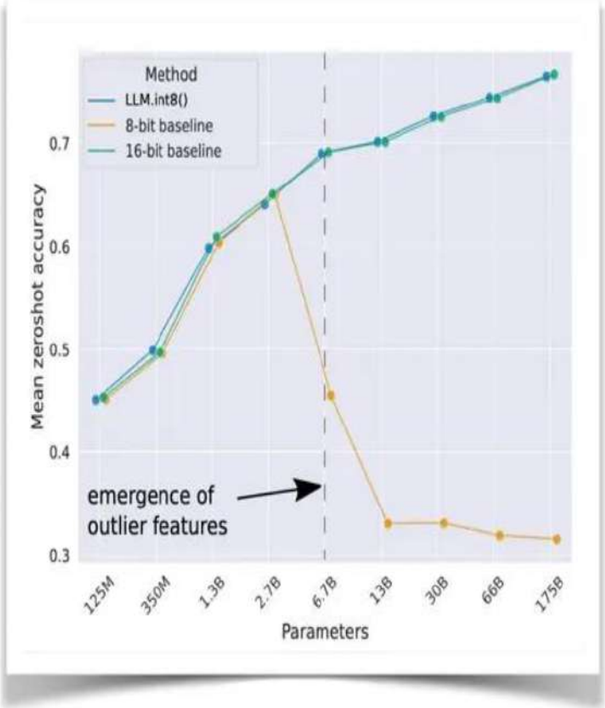
Group-wise quantization

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

bitsandbytes

https://arxiv.org/abs/2208.07339 (11/2022) + https://huggingface.co/blog/hf-bitsandbytes-integration

- Dynamic post-training quantization
- Quantization primitives for INT8/INT4 linear layers (weights only), and INT8 optimizers
- "*We find that outlier features strongly affect attention and the overall predictive performance of transformers. While up to 150k outliers exist per 2048 token sequence for a 13B model, these outlier features are highly systematic and only representing at most 7 unique feature dimensions.*"
- Vector-wise quantization: use different scaling factors when multiplying input and weight tensors
- Outlier dimensions are quantized separately with mixed-precision decomposition (16-bit)
- 8-bit quantization vs FP16: 2x memory savings, accuracy on par, can be faster for LLMs
- Mixed-precision decomposition is difficult to implement efficiently on hardware accelerators
- Integrated in `transformers`, `accelerate`, `peft`, and TGI



Parameters	125M	1.3B	2.7B	6.7B	13B
32-bit Float	25.65	15.91	14.43	13.30	12.45
Int8 absmax	87.76	16.55	15.11	14.59	19.08
Int8 zeropoint	56.66	16.24	14.76	13.49	13.94
Int8 absmax row-wise	30.93	17.08	15.24	14.13	16.49
Int8 absmax vector-wise	35.84	16.82	14.98	14.13	16.48
Int8 zeropoint vector-wise	25.72	15.94	14.36	13.38	13.47
Int8 absmax row-wise + decomposition	30.76	16.19	14.65	13.25	12.46
Absmax LLM.int8() (vector-wise + decomp)	25.83	15.93	14.44	**13.24**	**12.45**
Zeropoint LLM.int8() (vector-wise + decomp)	**25.69**	**15.92**	**14.43**	**13.24**	**12.45**

C4 validation perplexities on OPT models (lower is better)

GPT-3 Size Model dimension	Small 768	Medium 1024	Large 1536	XL 2048	2.7B 2560	6.7B 4096	13B 5140	175B 12288
FP16-bit baseline	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x
Int8 without overhead	0.99x	1.08x	1.43x	1.61x	1.63x	1.67x	2.13x	2.29x
Absmax PyTorch+NVIDIA	0.25x	0.24x	0.36x	0.45x	0.53x	0.70x	0.96x	1.50x
Vector-wise PyTorch+NVIDIA	0.21x	0.22x	0.33x	0.41x	0.50x	0.65x	0.91x	1.50x
Vector-wise	**0.43x**	**0.49x**	**0.74x**	**0.91x**	**0.94x**	**1.18x**	**1.59x**	**2.00x**
LLM.int8() (vector-wise+decomp)	0.14x	0.20x	0.36x	0.51x	0.64x	0.86x	1.22x	1.81x

Inference speedup (higher is better)

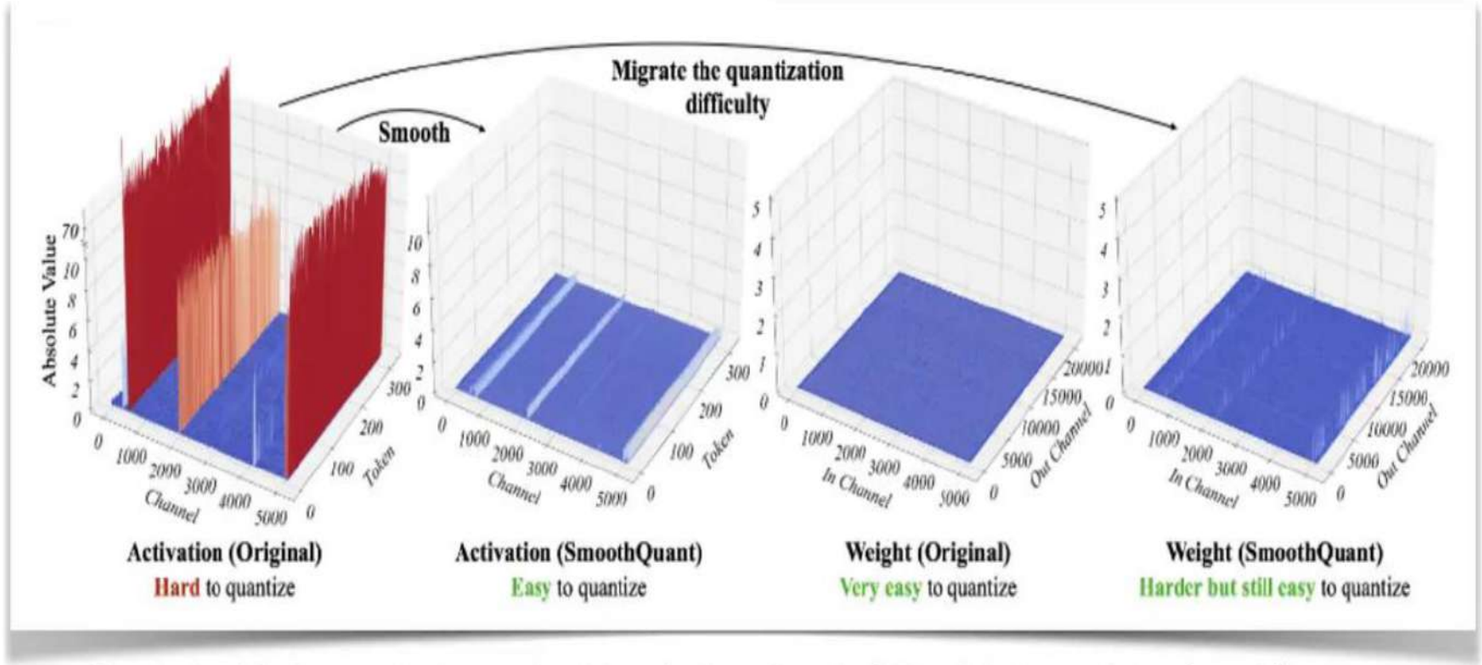
```
from transformers import AutoModelForCausalLM

model_8bit = AutoModelForCausalLM.from_pretrained(
    "bigscience/bloom-1b7", device_map="auto", load_in_8bit=True)
```

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

SmoothQuant

https://arxiv.org/abs/2211.10438 (11/2022) + https://github.com/mit-han-lab/smoothquant

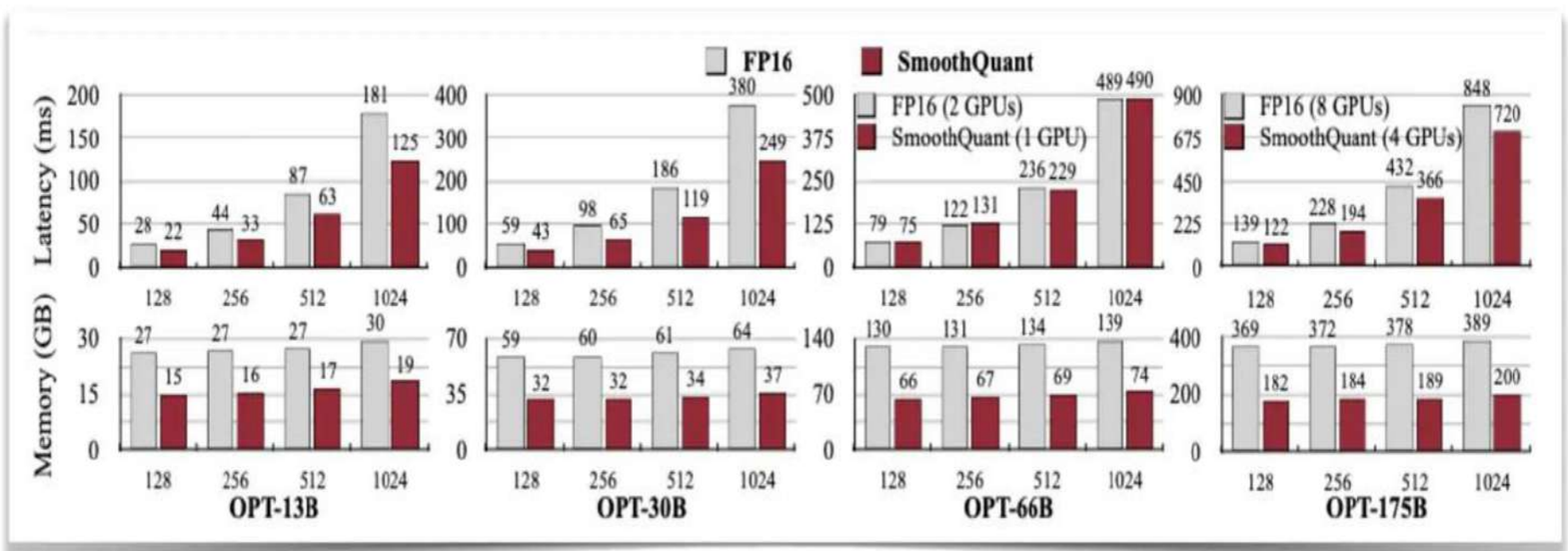
- Dynamic or static post-training quantization
- Activations are harder to quantize (much larger range)
- Migrate the quantization difficulty to the weight layers
- Rescale weights to reduce the magnitude of the activation ranges
  - Rescale activations of self-attention and linear layers
  - Quantize linear layers to INT8
- As accurate as bitsandbytes, much faster



Magnitude of the input activations and weights of a linear layer in OPT-13B before and after SmoothQuant.

Method	OPT-175B	BLOOM-176B	GLM-130B*
FP16	71.6%	68.2%	73.8%
W8A8	32.3%	64.2%	26.9%
ZeroQuant	31.7%	67.4%	26.7%
`LLM.int8()`	71.4%	68.0%	73.8%
Outlier Suppression	31.7%	54.1%	63.5%
SmoothQuant-O1	**71.2%**	68.3%	**73.7%**
SmoothQuant-O2	71.1%	**68.4%**	72.5%
SmoothQuant-O3	71.1%	67.4%	72.8%

Average accuracy
on WinoGrande, HellaSwag, PIQA, and LAMBADA



Inference latency (top) and memory usage (bottom) of the FasterTransformer implementation on NVIDIA A100-80GB GPUs

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Group-wise Precision Tuning Quantization (GPTQ)

https://arxiv.org/abs/2210.17323 (03/2023) + https://github.com/AutoGPTQ/AutoGPTQ
https://huggingface.co/blog/gptq-integration + https://huggingface.co/blog/overview-quantization-transformers

- Static post-training quantization
- New levels of quantization: 8, 4, 3, or 2-bit precision (weights only)
- Row-wise quantization (aka per-channel), plus modified Optimal Brain Quantization https://arxiv.org/abs/2208.11580 (08/2022)
- 4-bit quantization vs. FP16: 4x memory savings, same inference speed, negligible perf degradation
- The `exllamav2` kernels are enabled by default for faster inference https://github.com/turboderp/exllamav2
- Integrated in `transformers`, `accelerate`, `peft`, and TGI

Method	Bits	OPT-175B				BLOOM-176B			
		Wiki2	PTB	C4	LAMB. ↑	Wiki2	PTB	C4	LAMB. ↑
Baseline	16	8.34	12.01	10.13	75.59	8.11	14.59	11.71	67.40
RTN	4	10.54	14.22	11.61	71.34	8.37	15.00	12.04	66.70
GPTQ	4	**8.37**	**12.26**	**10.28**	**76.80**	**8.21**	**14.75**	**11.81**	**67.71**
RTN	3	7.3e3	8.0e3	4.6e3	0	571.	107.	598.	0.17
GPTQ	3	**8.68**	**12.68**	**10.67**	**76.19**	**8.64**	**15.57**	**12.27**	**65.10**
GPTQ	3/g1024	8.45	12.48	10.47	77.39	8.35	15.01	11.98	67.47
GPTQ	3/g128	8.45	12.37	10.36	76.42	8.26	14.89	11.85	67.86

```
from transformers import AutoModelForCausalLM, AutoTokenizer, GPTQConfig

model_id = "facebook/opt-125m"
tokenizer = AutoTokenizer.from_pretrained(model_id)

quantization_config = GPTQConfig(bits=4, dataset="c4", tokenizer=tokenizer)

# Quantize model
model = AutoModelForCausalLM.from_pretrained(model_id,
device_map="auto",quantization_config=quantization_config)
```

INT3 OPT-175B
average per-token latency
batch size 1, sequence length 128

GPU	FP16	3bit	Speedup	GPU reduction
A6000 – 48GB	589ms	130ms	4.53×	8 → 2
A100 – 80GB	230ms	71ms	3.24×	5 → 1

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

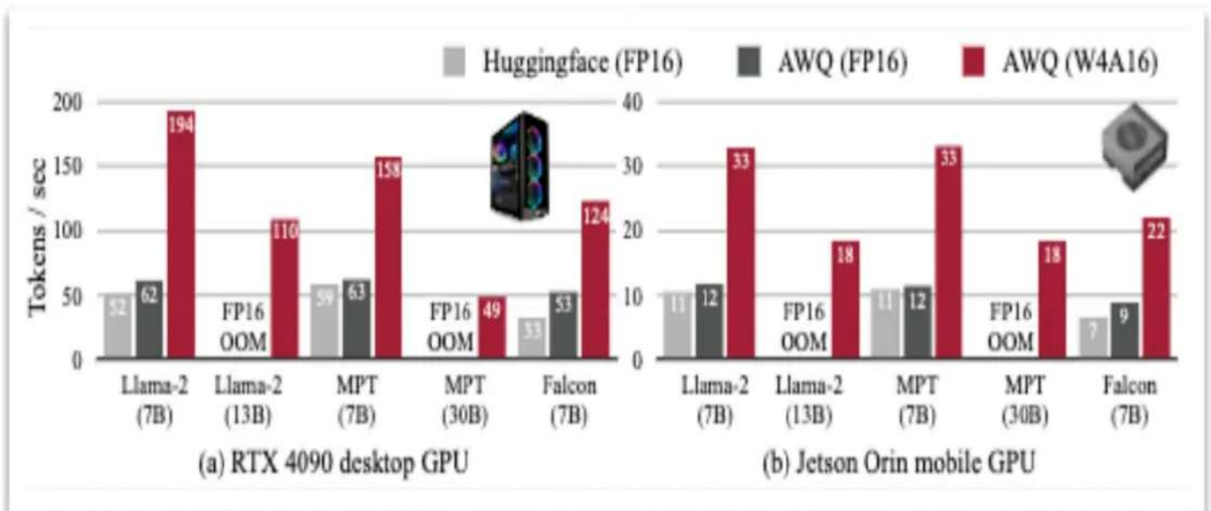
Activation-aware Weight Quantization (AWQ)

https://arxiv.org/abs/2306.00978 (10/2023)

- Static post-training quantization
- 0.1-1% of weights have a critical influence: they are not quantized to minimize degradation
- These salient weights are identified by looking at activations
- AWQ needs 10x less calibration data than GPTQ, and is more robust across datasets
- AWQ performs well on instruction-tuned and multimodal models
- At the moment, `transformers` supports model-loading only
- Models can be quantized with https://github.com/mit-han-lab/llm-awq or https://github.com/casper-hansen/AutoAWQ

PPL↓		Llama-2			LLaMA			
		7B	13B	70B	7B	13B	30B	65B
FP16	-	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128	RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
	GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
	GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
	AWQ	**6.24**	**5.32**	**3.74**	**6.35**	**5.52**	**4.61**	**3.95**
INT4 g128	RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
	GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
	GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
	AWQ	**5.60**	**4.97**	**3.41**	**5.78**	**5.19**	**4.21**	**3.62**

Perplexity (lower is better)



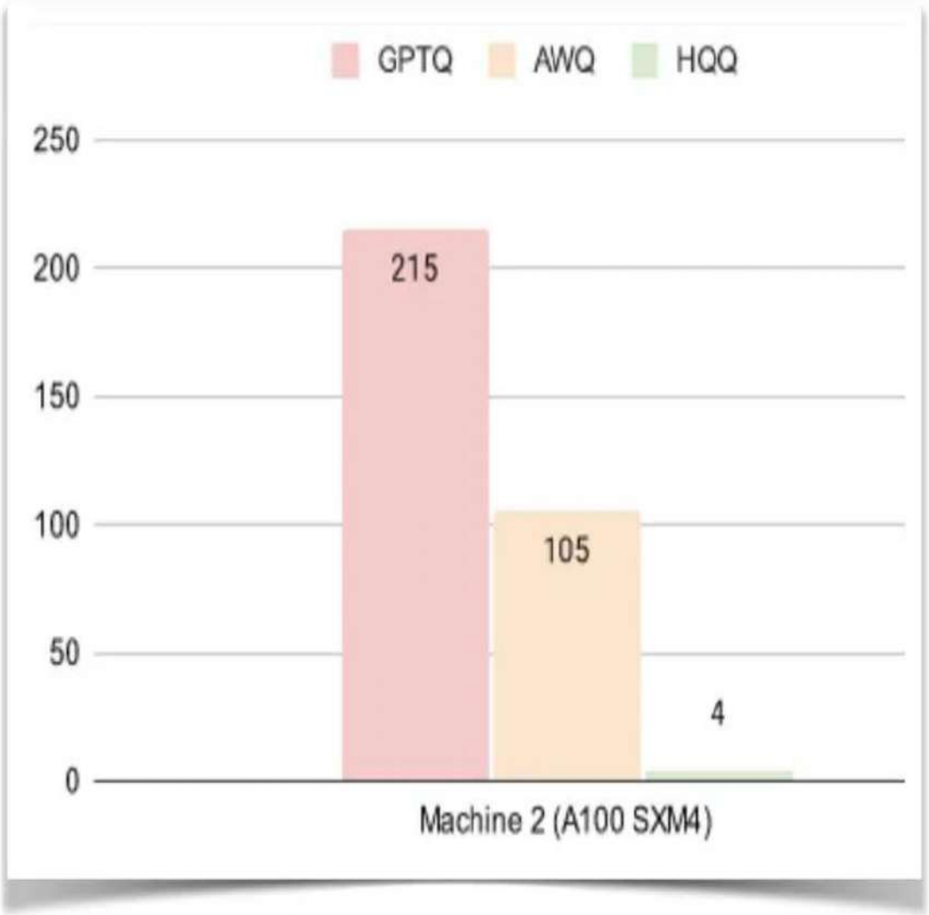
AWQ is up to 3.9× than the FP16 implementation on an NVIDIA 4090 GPU

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Half-quadratic Quantization (HQQ)

https://mobiusml.github.io/hqq_blog/ + https://github.com/mobiusml/hqq + https://huggingface.co/mobiuslabsgmbh (12/2023)

- Dynamic post-training quantization
- Same accuracy as static PTQ, without the calibration phase
- Minimize error in weight quantization and model outliers
- Outperforms GPTQ and AWQ on most tests



Llama2-70B quantization time (minutes)

Method	nBits	Llama-2-7B		Llama-2-13B		Llama-2-70B	
		PPL ↓	MEM ↓	PPL ↓	MEM ↓	PPL ↓	MEM ↓
FP	16	5.18	13.5	**4.63**	25.6	OOM	OOM
BNB	8	5.22	7.9	4.67	14.4	3.17	**68.15**
GPTQ_g128	8	**5.19**	7.8	**4.63**	14.8	**3.12**	74.87
HQQ_g128	8	**5.19**	**7.6**	**4.63**	**14**	**3.12**	69.32
BNB_g64	4	5.43	4.7	4.79	8.2	3.29	39.11
GPTQ_g128	4	5.41	5	4.74	8.9	3.24	40
GPTQ_g64	4	5.38	5	4.73	9.1	3.23	41.13
AWQ_g128	4	5.32	**4.6**	4.71	8.2	3.21	**35.78**
AWQ_g64	4	**5.28**	**4.6**	**4.7**	8.5	3.2	37.08
HQQ_g128	4	5.35	**4.6**	4.74	**7.9**	3.21	35.97
HQQ_g64	4	5.3	**4.6**	**4.7**	8.2	**3.19**	37.52
GPTQ_g128	3	6.3	3.9	5.25	7	3.85	33.7
GPTQ_g64	3	6.1	4	5.16	7.3	3.7	33.47
HQQ_g128	3	6.2	**3.8**	5.15	**6.8**	3.58	**30.11**
HQQ_g64	3	**5.82**	4.5	**4.98**	7.4	**3.45**	33.46
GPTQ_g64	2	nan	**3.5**	13	6	9.44	24.5
HQQ_g32	2	15.61	**3.5**	7.63	**5.9**	4.82	**24.2**
HQQ_g16	2	**7.3**	4.1	**6.36**	6.9	**4.12**	30.27
HQQ_g16_s*	2	7.31	3.7	6.37	6.1	4.13	26.37

Perplexity on wikitext-2 and memory usage (lower is better)

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

Optimum Intel

https://github.com/huggingface/optimum-intel + https://huggingface.co/docs/optimum
https://github.com/huggingface/optimum-intel/tree/main/examples

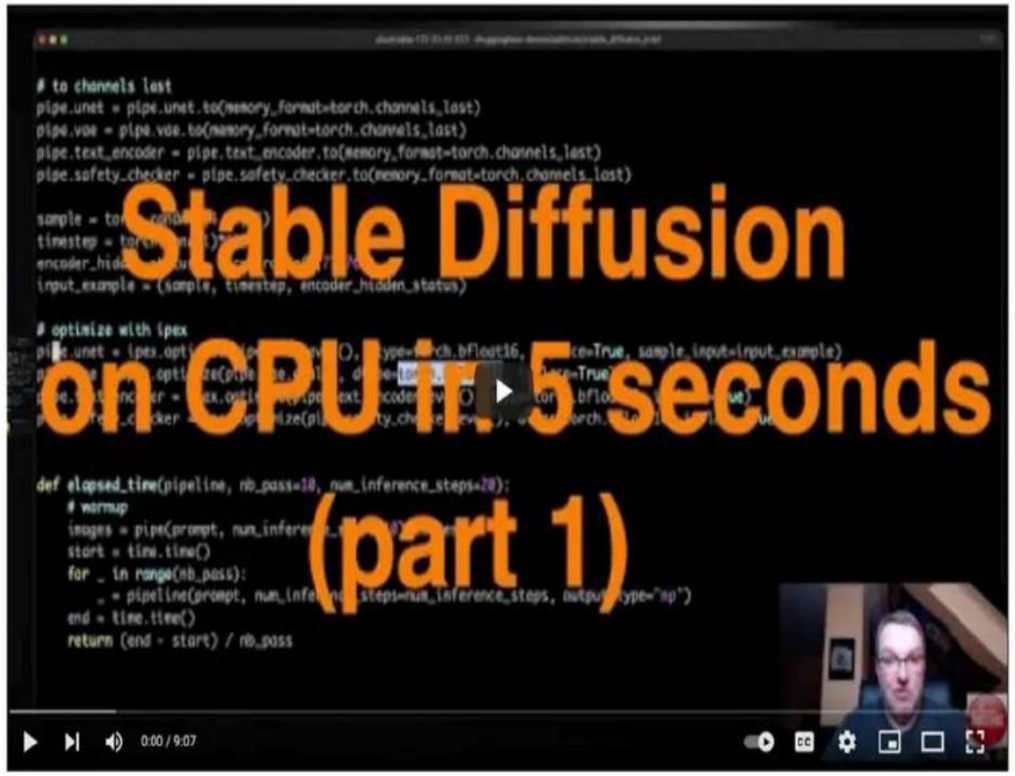
- Accelerate Hugging Face models on Intel architectures
- Simple transformers-like API and CLI for:
  - Intel Neural Compressor https://github.com/intel/neural-compressor
  - Intel OpenVINO https://github.com/openvinotoolkit/openvino
- Static and dynamic post-training quantization, quantization-aware training
- Different recipes are supported, including SmoothQuant

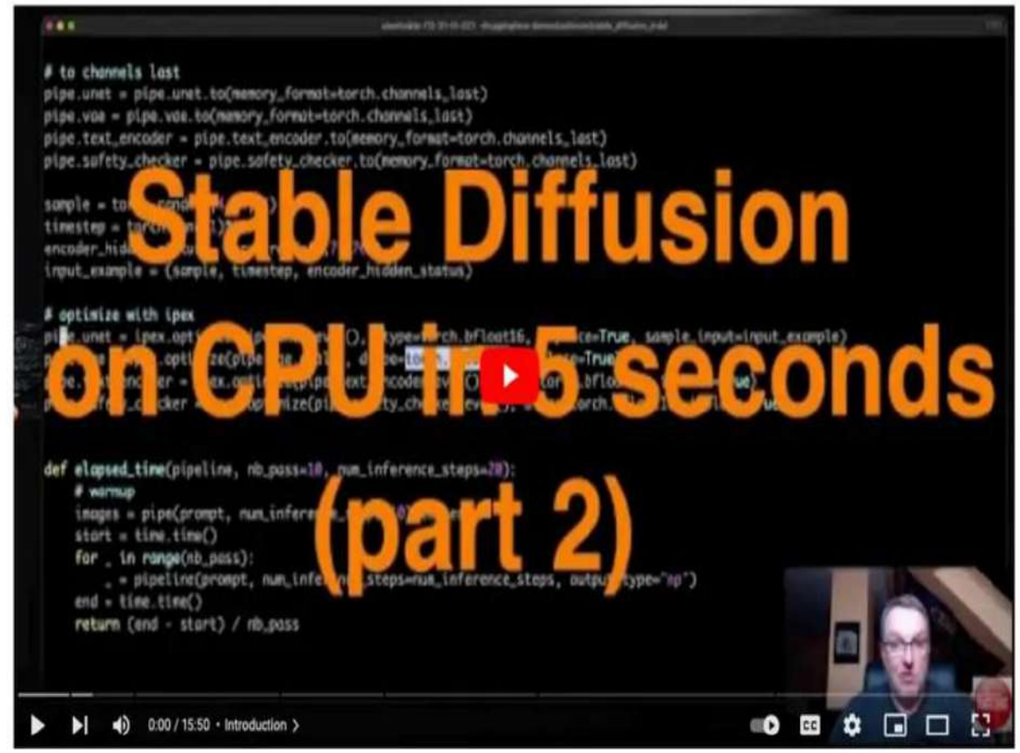
```
from optimum.intel import INCQuantizer
from neural_compressor.config import AccuracyCriterion, TuningCriterion, PostTrainingQuantConfig

# Set the accepted accuracy loss to 5%
accuracy_criterion = AccuracyCriterion(tolerable_loss=0.05)
# Set the maximum number of trials to 10
tuning_criterion = TuningCriterion(max_trials=10)
quantization_config = PostTrainingQuantConfig(
    approach="dynamic", accuracy_criterion=accuracy_criterion, tuning_criterion=tuning_criterion
)
quantizer = INCQuantizer.from_pretrained(model, eval_fn=eval_fn)
quantizer.quantize(quantization_config=quantization_config,
save_directory="dynamic_quantization")
```

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.

CPU

Demo: Accelerating Stable Diffusion Inference with Intel OpenVINO, IPEX and Hugging Face





https://www.youtube.com/watch?v=KJDCGyZ2fPw

https://www.youtube.com/watch?v=78gE0CHxDbo

The author of this material is Julien Simon https://www.linkedin.com/in/juliensimon unless explicitly mentioned.
This material is shared under the CC BY-NC 4.0 license https://creativecommons.org/licenses/by-nc/4.0/
You are free to share and adapt this material, provided that you give appropriate credit, provide a link to the license, and indicate if changes were made.
You may not use the material for commercial purposes. You may not apply any restriction on what the license permits.